

robot_ros2_RTK

ROS 2 Jazzy workspace for the Hydra-based RTK robot setup. The repository is a monorepo: all packages needed by this workspace live under `src/`, so a fresh clone can be built without restoring nested git repositories.

Quick Start

On the robot computer:

```
cd ~/ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch hydra_ros camera_yolo_orangepi.launch.py
```

On the laptop connected to the same local network:

```
cd ~/ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch hydra_visualizer laptop_rviz.launch.yaml
```

The laptop RViz subscribes only to the scene graph and a low-rate annotated YOLO image. It does not subscribe to the full camera stream.

First Build On A New Machine

Install the base dependencies:

```
sudo apt update
sudo apt install -y python3-rosdep python3-vcstool ros-dev-tools
sudo apt install -y ros-jazzy-gtsam nlohmann-json3-dev pybind11-dev
sudo apt install -y libgoogle-glog-dev libgtest-dev libyaml-cpp-dev libzmqpp-dev
```

Initialize `rosdep` if it has not been initialized yet:

```
sudo rosdep init
rosdep update
```

Install package dependencies and build:

```
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y --rosdistro jazzy
colcon build --symlink-install
source install/setup.bash
```

`kimera\_rpgo`, `kimera\_pgmo`, and `hydra` require GTSAM. On Ubuntu 24.04 with ROS Jazzy this is provided by `ros-jazzy-gtsam`.

External YOLO Workspace

The robot launch files expect the YOLO segmentation workspace at:

```
~/ros2_yolo_ws
```

The executable script used by the launches is:

```
~/ros2_yolo_ws/src/yolo_segmentation/yolo_segmentation/segmentation_node_floor.py
```

If that workspace is somewhere else, pass `yolo\_ws:=/path/to/ros2\_yolo\_ws` to `camera\_yolo\_orangepi.launch.py` or `bag\_yolo\_orangepi.launch.py`.

Launch Files

`hydra\_ros camera\_yolo\_orangepi.launch.py`

Main live-robot launch. It starts:

- Intel RealSense camera at `424x240x15`.

- RealSense gyro/accelerometer and ``imu_filter_madgwick``.
- ``rtabmap_odom/rgbd_odometry`` with IMU initialization enabled.
- Static transforms for ``camera_link -> base_link_gt`` and ``map -> odom``.
- YOLO segmentation from ``~/ros2_yolo_ws``.
- ``hydra.launch.yaml`` after the camera, odometry, TF, and YOLO nodes have had time to start.

Use it on the robot computer:

```
ros2 launch hydra_ros camera_yolo_orangepi.launch.py
```

``hydra_ros bag_yolo_orangepi.launch.py``

Offline/debug launch for a rosbag. It starts rosbag playback, odometry, static TF, YOLO segmentation, and Hydra in the same topic layout as the live camera launch.

Default bag path:

```
proezd15_05V4
```

Example:

```
ros2 launch hydra_ros bag_yolo_orangepi.launch.py bag_path:=/path/to/bag
```

``hydra_ros hydra.launch.yaml``

Core Hydra launch. It starts the Hydra node, the Hydra visualizer, local RViz by default, and two low-rate RViz image republishers.

Important output topics:

```
/hydra_visualizer/graph
/hydra_visualizer/agent_poses
/rviz/camera/color/image_raw
/rviz/yolo_segmentation/result
```

The graph is published continuously. Camera processing for Hydra stays on the full-rate camera stream:

```
/camera/camera/color/image_raw  
/camera/camera/aligned_depth_to_color/image_raw  
/yolo_segmentation/labels
```

Only the RViz image topics are throttled to 1 Hz:

```
/camera/camera/color/image_raw    -> /rviz/camera/color/image_raw  
/yolo_segmentation/result         -> /rviz/yolo_segmentation/result
```

Useful arguments:

```
start_visualizer:=true|false  
start_rviz:=true|false  
start_rviz_image_throttle:=true|false  
rviz_image_rate_hz:=1.0  
start_rviz_yolo_image_throttle:=true|false  
rviz_yolo_image_rate_hz:=1.0
```

`hydra\_visualizer laptop\_rviz.launch.yaml`

Lightweight RViz launch for the laptop. It opens:

```
src/hydra_ros/hydra_visualizer/rviz/laptop_streaming_visualizer.rviz
```

The laptop config subscribes to:

```
/hydra_visualizer/graph  
/hydra_visualizer/agent_poses  
/rviz/yolo_segmentation/result
```

This is the recommended laptop-side command:

```
ros2 launch hydra_visualizer laptop_rviz.launch.yaml
```

``hydra_visualizer streaming_visualizer.launch.yaml``

Full/local visualizer launch used by Hydra on the robot computer. It keeps the normal local RViz behavior, including mesh and local image displays. Use this through ``hydra.launch.yaml`` unless you are debugging the visualizer directly.

Project Structure

```
.
├─ colcon_defaults.yaml
├─ README.md
├─ src/
│   ├─ hydra/                # Core Hydra library and configs
│   ├─ hydra_ros/
│   │   ├─ hydra_ros/        # ROS wrapper, robot/bag launches, throttler
│   │   ├─ hydra_visualizer/ # Hydra visualizer, RViz configs, laptop launch
│   │   └─ hydra_msgs/       # Hydra ROS messages/services
│   ├─ kimera_pgmo/          # Kimera-PGMO and RViz/ROS integration
│   ├─ kimera_rpgo/          # Pose graph optimization dependency
│   ├─ pose_graph_tools/     # Pose graph messages/tools
│   ├─ spark_dsg/            # Dynamic scene graph library
│   ├─ spatial_hash/         # Spatial hashing dependency
│   ├─ config_utilities/     # Config parsing utilities
│   ├─ semantic_inference/   # Semantic inference packages/messages
│   ├─ realsense-ros/        # RealSense ROS packages
│   ├─ realsense2_ros_mqtt_bridge/ # RealSense bridge package
│   ├─ security_system/      # Additional camera/YOLO utilities
│   ├─ kursant1_driver/      # Robot driver package
│   ├─ kursant1_task_processor/ # Robot task processing package
│   ├─ ianvs/                # Additional dependency package
│   └─ teaser_plusplus/      # TEASER++ dependency
└─ install/, build/, log/    # Generated by colcon, not part of source
```

The YOLO segmentation workspace is intentionally outside this repository:

```
~/ros2_yolo_ws
```

Network Visualization Model

The important rule is:

- Hydra, odometry, and graph construction use full-rate local camera/depth/YOLO data on the robot computer.
- The laptop receives the scene graph continuously.
- The laptop receives only one annotated YOLO image per second.

This avoids the old failure mode where laptop RViz could subscribe to the raw camera topic and overload the local network.

Check the expected rates:

```
ros2 topic hz /hydra_visualizer/graph
ros2 topic hz /rviz/yolo_segmentation/result
ros2 topic hz /rviz/camera/color/image_raw
```

The two `/rviz/...image...` topics should be near `1 Hz`.

Changes Since The Init Commit

Relative to the initial workspace snapshot (`21ad9eb`), the repository now has:

- A monorepo layout: nested package repositories were flattened into normal source directories under `src/`.
- Root-level build and setup documentation.
- `camera_yolo_orangepi.launch.py` for live RealSense + IMU + RTAB-Map odometry + YOLO + Hydra startup.
- `bag_yolo_orangepi.launch.py` for rosbag replay with the same processing chain.
- `throttled_image_republisher.py`, installed by `hydra_ros`, for low-bandwidth RViz image topics.

- ``laptop_rviz.launch.yaml`` and ``laptop_streaming_visualizer.rviz`` for remote laptop visualization.
- ``hydra.launch.yaml`` arguments and nodes for 1 Hz RGB and YOLO annotated image streams: - ``/rviz/camera/color/image_raw`` - ``/rviz/yolo_segmentation/result``
- Restored IMU-based initialization for camera odometry after testing showed that disabling it could prevent fresh poses and stop graph updates.

Troubleshooting

If the graph is empty in RViz, first check that odometry and TF are alive:

```
ros2 topic echo /rtabmap/odom --once
ros2 run tf2_ros tf2_echo odom camera_link
ros2 topic hz /hydra_visualizer/graph
```

If the laptop image is too fast or missing:

```
ros2 topic hz /rviz/yolo_segmentation/result
ros2 topic info /rviz/yolo_segmentation/result --verbose
```

If the laptop clone fails with missing ``GTSAMConfig.cmake``, install:

```
sudo apt install -y ros-jazzy-gtsam
```